实 验 报 告

实验名称：实验五 HMAX 模型实现

课程名称：认知科学与类脑识别

实验地点：K2

实验日期：2024.10.16

班 级：2022 级人工智能班

姓 名：薛润泽

学 号：202200130077

一 实验目的：

加深对 HMAX 模型的理解，能够使用 HMAX 模型解决简单问题

二 实验环境：

windows

三 实验内容：

根据 HMAX 模型的相关知识，使用 Python 语言实现一个简单的 HMAX 模型。

要求：

（1） 下载 MNIST 数据集。

（2） 构建 HMAX 模型。

（3） 使用 MNIST 数据集中的训练集训练网络，使用测试集测试训练好的网络。

四 实验步骤：

实验程序：

（1）完成对本地数据集的加载

```
import numpy as np
import gzip
from scipy.ndimage import convolve
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelBinarizer
import matplotlib.font_manager as fm

def load_mnist_images(file_path):
    with gzip.open(file_path, 'rb') as f:
        data = np.frombuffer(f.read(), np.uint8, offset=16)
    return data.reshape(-1, 28, 28)

def load_mnist_labels(file_path):
    with gzip.open(file_path, 'rb') as f:
        data = np.frombuffer(f.read(), np.uint8, offset=8)
    return data

def load_mnist():
    x_train = load_mnist_images('MNIST/raw/train-images-idx3-ubyte.gz')
    y_train = load_mnist_labels('MNIST/raw/train-labels-idx1-ubyte.gz')
    x_test = load_mnist_images('MNIST/raw/t10k-images-idx3-ubyte.gz')
    y_test = load_mnist_labels('MNIST/raw/t10k-labels-idx1-ubyte.gz')
    return (x_train, y_train), (x_test, y_test)
```

```
(x_train, y_train), (x_test, y_test) = load_mnist()
```

（2）定义 gabor 函数，gabor 函数实现了 Gabor 滤波器，可以从不同尺度和方向对图像进行滤波，很好地模拟视觉皮层简单细胞对图像的空间频率和方向选择性，从而提取图像的特征信息。

```
from scipy.ndimage import convolve

def gabor_filter(img, ksize, sigma, theta, lambd, gamma, psi):
    y, x = np.meshgrid(np.linspace(-1, 1, ksize), np.linspace(-1, 1, ksize))
    x_theta = x * np.cos(theta) + y * np.sin(theta)
    y_theta = -x * np.sin(theta) + y * np.cos(theta)
    gb = np.exp(-0.5 * (x_theta**2 + gamma**2 * y_theta**2) / sigma**2) * np.cos(2 * np.pi * x_theta / lambd + psi)
    return convolve(img, gb)
```

（3）定义 S1 层计算函数，S1 层是 HMAX 模型的第一个处理层，主要是利用 Gabor 滤波器来对图像进行特征提取。在这里根据实验要求，对 0,45,90,135 这 4 个方向定义了 8 个尺度，结合 2 个分级，共计一个图像会提取 64 张特征图。

```python
def s1_layer(img):
    scales = [3, 5, 7, 9, 11, 13, 15, 17]
    directions = [0, 45, 90, 135]
    responses = []
    for scale in scales:
        for direction in directions:
            theta = np.deg2rad(direction)
            response = gabor_filter(img, scale, 4.0, theta, 10.0, 0.5, 0)
            responses.append(response)
    return np.array(responses)
```

✓ 0.0s

（4）定义最大池化函数，最大池化函数用于降低特征图的维度的同时保留最重要信息，从而减小训练中的计算量。

```python
def max_pooling(responses, pool_size):
    pooled_responses = []
    for response in responses:
        pool_h, pool_w = pool_size
        pooled = response.reshape(response.shape[0] // pool_h, pool_h, response.shape[1] // pool_w, pool_w).max(axis=(1, 3))
        pooled_responses.append(pooled)
    return np.array(pooled_responses)
```

（5）定义 C1 层计算函数，C1 层即利用最大池化函数完成对提取特征的池化

```python
def c1_layer(responses, pool_size=(4, 4)):
    pooled_responses = max_pooling(responses, pool_size)
    return pooled_responses
```

（6）定义 S2 层计算函数，S2 层模拟复杂细胞层，主要用于对来自 C1 层的特征图进行整合，使用随机滤波器进行滤波。

```python
def s2_layer(c1_responses, patches):
    s2_responses = []
    for patch in patches:
        patch_responses = []
        for c1_response in c1_responses:
            response = np.linalg.norm(c1_response - patch)
            patch_responses.append(response)
        s2_responses.append(np.array(patch_responses))
    return np.array(s2_responses)
```

✓ 0.0s

（7）定义 C2 层计算函数，类似与 C1 层，C2 层对 S2 层的输出进行最大池化，完成全局特征提取、特征选择和降维。

```
def c2_layer(s2_responses):
    c2_responses = []
    for responses in s2_responses:
        max_vals = np.max(responses, axis=0)
        c2_responses.append(max_vals)
    return np.array(c2_responses).flatten()
```

（8）由于使用全部样本训练时间代价过高，所以使用随机抽样的方法来进行训练，这里选择抽取样本 1000 进行训练

```
def random_select_data(x_data, y_data, sample_size):
    indices = np.random.choice(len(x_data), sample_size, replace=False)
    x_sample = x_data[indices]
    y_sample = y_data[indices]
    return x_sample, y_sample

# 从训练集和测试集中随机选择1000个样本
sample_size = 1000
x_train_sample, y_train_sample = random_select_data(x_train, y_train, sample_size)
x_test_sample, y_test_sample = random_select_data(x_test, y_test, sample_size)
```

✓ 0.0s

（9）定义特征提取函数

```
# 特征提取
def extract_features(images, patches):
    features = []
    for img in images:
        s1 = s1_layer(img)
        c1 = c1_layer(s1, pool_size=(4, 4))  # 调整池化尺寸
        s2 = s2_layer(c1, patches)
        c2 = c2_layer(s2)
        features.append(c2)
    return np.array(features)
```

（10）对训练集和测试集进行特征提取

```
x_train_features = extract_features(x_train_sample, patches)
```
✓ 9.4s

```
x_test_features = extract_features(x_test_sample, patches)
```
✓ 9.5s

（11）使用 SVM 进行分类预测

```
# 使用训练数据和特征提取初始化SVM分类器
clf = SVC(kernel='linear', probability=True)
losses = []
label_binarizer = LabelBinarizer()
```
✓ 0.0s

（12）进行训练

```
for i in range(len(x_train_features)):
    x_batch = x_train_features[:i+1]
    y_batch = y_train_sample[:i+1]
    if len(np.unique(y_batch)) < 2:
        continue  # 跳过仅包含一个类的批次
    # label_binarizer = LabelBinarizer()
    y_batch_bin = label_binarizer.fit_transform(y_batch)
    clf.fit(x_batch, y_batch)
    probas = clf.predict_proba(x_batch)

    # 计算损失值
    loss = -np.sum(np.log(probas) * y_batch_bin) / y_batch.shape[0]
    losses.append(loss)
```

（13）进行预测，并将预测结果写入到文档中

```
# 进行预测
predictions = clf.predict(x_test_features)
accuracy = accuracy_score(y_test_sample, predictions)

# 将结果写入文件
with open('results.txt', 'w') as f:
    f.write(f'准确率: {accuracy}\n')
```

1. 程序实验结果：

使用随机 1000 样本程序训练结果如下图：



可以看到使用，训练预测正确率为 0.258，对于有 10 个预测结果的手写数据集来说，随机预测正确率为 0.1。模型正确率 0.258>随机正确率 0.1，模型有一定效果，但是不够好。

2. 分析和改进：

分析到，可能是因为随机数据提取样本过少，导致模型训练正确率低。

使用随机样本，并调整模型训练参数后，训练结果如下图：



模型准确率大大提高。

五 实验小结：

本实验使用 HMAX 模型，根据人脑是由层级结构处理视觉信息的生物知识，通过建立 S1 层，C1 层，S2 层，C2 层模拟了人类视网膜的简单细胞层和视觉皮层的复杂细胞层。最后将处理信息传递到具有大感受野的 VTC 细胞层，从而得到输入信息的分类。

本实验训练比较慢，可以采取随机抽样法加快进度，但是准确率会降低。